

What Is the Call Stack?

Programs use stacks too. The program's *call stack*, also simply called *the stack*, is a stack of frame objects. *Frame objects*, also simply called *frames*, contain information about a single function call, including which line of code called the function, so the execution can move back there when the function returns.

Frame objects are created and pushed onto the stack when a function is called. When the function returns, that frame object is popped off the stack. If we call a function that calls a function that calls a function, the call stack will have three frame objects on the stack. When all these functions return, the call stack will have zero frame objects on the stack.

Programmers don't have to write code dealing with frame objects, since the programming language handles them automatically. Different programming languages have different ways of implementing frame objects, but in general they contain the following:

- The return address, or the spot in the program where the execution should move when the function returns
- The arguments passed to the function call
- A set of local variables created during the function call

For example, take a look at the following *localVariables.py* program, which has three functions, just as our previous *functionCalls.py* and *functionCalls.html* programs did:

Python

```
def a():
    ❶ spam = 'Ant'
    ❷ print('spam is ' + spam)
    ❸ b()
    print('spam is ' + spam)

def b():
    ❹ spam = 'Bobcat'
    print('spam is ' + spam)
```